Table of Contents

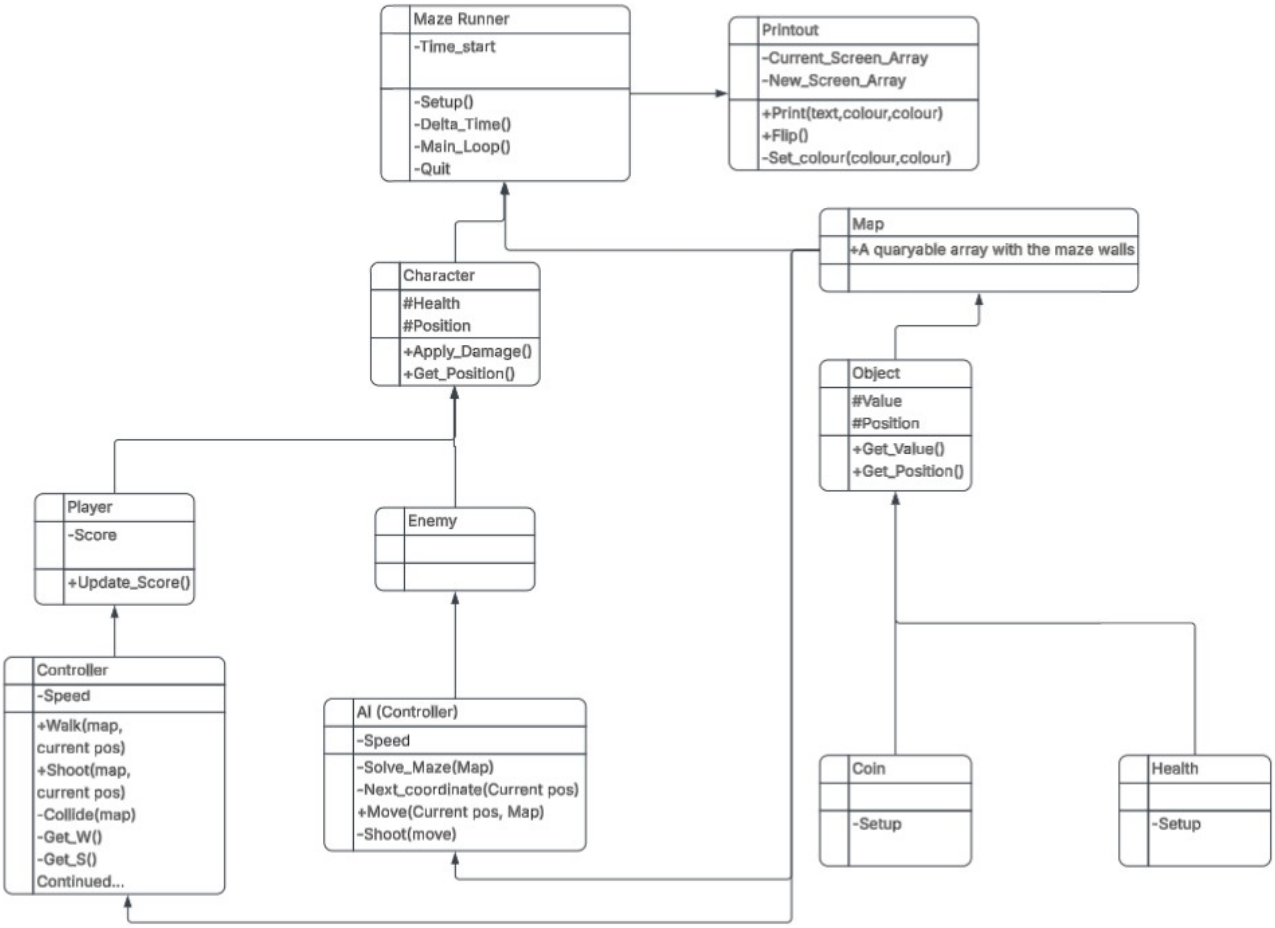

Figure 1: Initial Idea being simple with #inheritable, -Private, and +Public. only arrows due to not knowing how to change them.

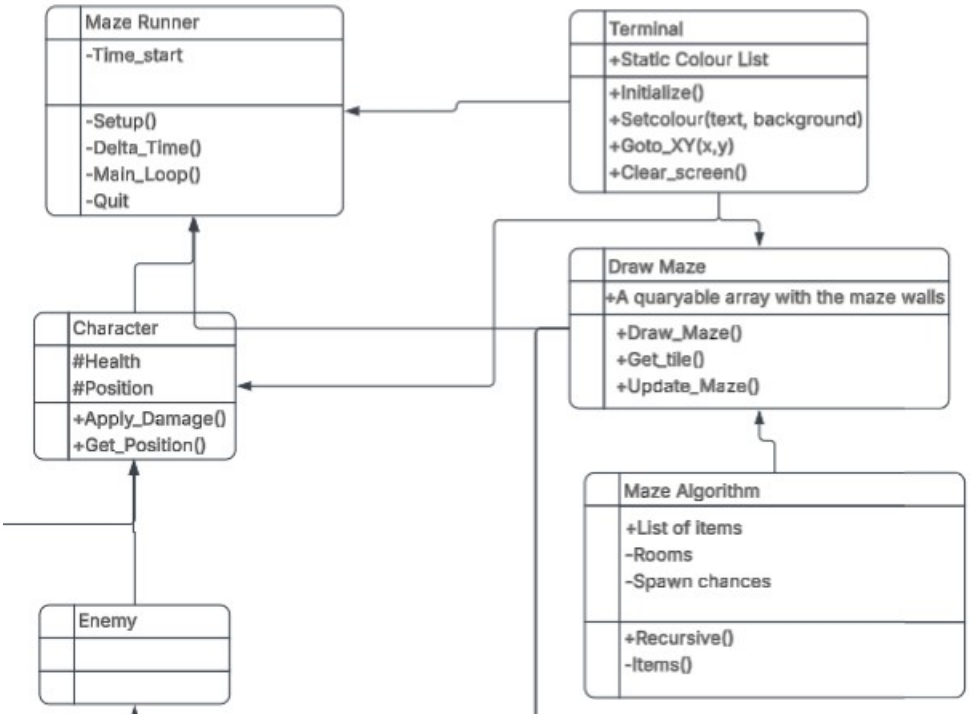

Figure 2: when the idea of a maze generation algorithm came ideas begun to pile ontop of the base design before it was done, bad idea.

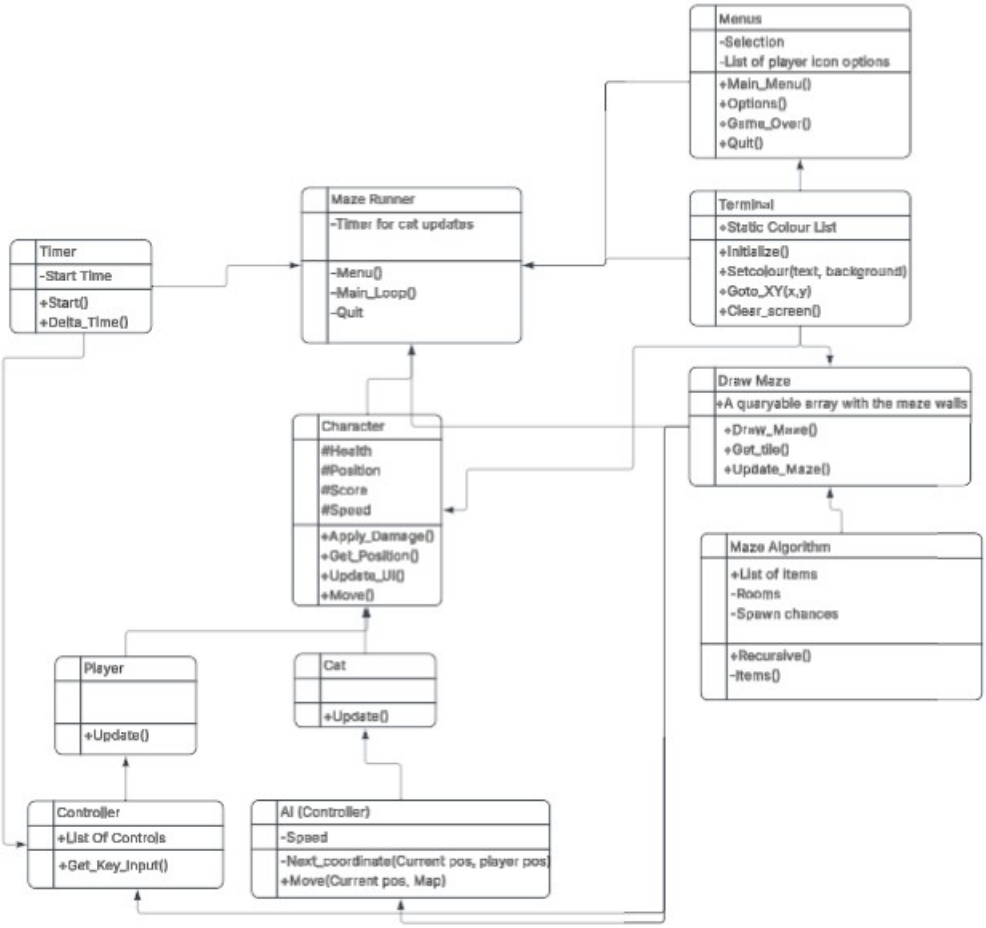

Figure 3: Simpler that the final code layout but when it started it got out of hand quickly like the character doing way too much.

Minor changes from the original assignment

Names

I could not see a major reason to change every variable name in the code from what it was as it is essentially the same game as the original but I did make the player side refer to it being a mouse game instead of my original title of maze runner.
A second reason for not changing a bunch of names is that I have found it to often break when renaming filenames and having to make a new file. And in addition it would be rather confusing to work on.

Major Changes

Maze

Originally the plan was to keep it simple with handmade mazes, this changed quickly due to my desire to keep the maze at 32*128 making it quicker to figure out my own algorithm. The bonus form this is that making infinite levels is easy and it can be expanded easily as well.

Screen

Original idea was to have a screen buffer system so that I could write everything into one big array like how the maze is stored and “Flip” the entire screen to avoid screen flickering with frequent updates. This worked well in the beginning before I got to adding colour and got stuck. I eventually ended up going for a single tile flip system instead where I print the previous tile under the thing that is going to move before printing it where it is going. This reduce flickering and stops me from needing to print the entire screen every time, but it ended up localized to whats moving which is a terrible way to do it in scale. If I were to do it now I would ensure that I had a class that handled the printing so it would be easier to work with.
After encountering a limitation with texturing I begun using utf-8 and a list from the consolas font to expand the artistic feel a lot.

Unable to figure out how to set the font and font size (I believe is due to command prompt/windows updates) that idea was also scrapped unfortunately since this could have made the grids to be square and fix the strange perspective.-

Movement

the original movement was simple but incredible. The problem was that the player didn’t have a class of any kind which isn’t good for OOP but it was in my opinion a better solution for the current project than the mess of a movement system I ended up cobbling togheter.
I had a plan to make projectiles so that the player could destroy weak spots in the wall to help quicken the navigation, but having problems making them appear the idea was scrapped and the weak spots became just holes in the wall instead. The idea of making the AI pathfind directly to the player would be easy enough but was scrapped due to the player having no way to avoid it due to the projectiles

being scrapped. A midpoint was made instead making the AI walk randomly but favor going towards the player slightly which could be increased some.

Addition

An unnecessary addition I still like is the ability to change the player icon on will from the options menu. This is done optimized due to problems figuring out how to step trough the symbols like you can with ascii. The idea of also choosing its colour was scrapped for no real reason other than feeling it even more unnecessary.

Issues

Maze

Because of the way I generate my maze it gets long stretches that can feel a bit empty. To combat this the idea of making different rooms emerged and a loot room was added with higher coin spawn rate and the likelihood of a heart that heals and increase max hp.
The rooms for hordes was scrapped due to the projectiles again, and the room for buying upgrades was discarded due to the complexity of balance and possible soft locks from missing abilities when trying to progress. A way around this would be to track abilities and only generate things they work on after they are bought.
An issue detected when finishing up is the maze algorithm has a chance of making rooms without doors that if unlucky could block off the exit. Therefore the spawnrate of extra doors was raised a lot higher to work as a temporary fix.
The planned safeguard for this problem never got implemented due to the projectile again. The idea was to use the same pathfindig for the enemies as for the maze so that you could verify a path from the start till the end before printing the maze and regenerate if something like this occurred.

Movement

The movement became strange after moving the delta time and movement out of the main loop and into classes. Different directions accelerate differently even though the code is (as far as I can see) identical besides one being subtracting from the position instead of adding to go the other way and the two others the same but on the other axis.

Pickup

When picking up items you go invisible until you move which was fixed and then reintroduced when the cat was added.

Why Do it this way?

Maze

Although I have made a mess of things with time due to plenty of mistakes I would still argue for how I've made my maze work on the ground level. I could easily have generated my maze then implement interactibles as individual classes and just generate them on top later my method does have an (possibly subjective) advantage with it's simplicity.
Done right and you don't have to worry about making collisions work since you can just query the location you want and don't need to know what it is unless you want to. By simply making a list and returning walkable or not you get a simple and easy collision system that would be easy to change and maintain (what I wish I did in retrospect).
If you take the time to put the moving objects on the map as well instead of on a level on top it would be easier to detect and interact with other moving objects. The only thing is that pickups that can be walked over and not picked up would have to be stored and replaced when moving away from it but making the array that stores everything 3 dimensional you could get something like a collision layer from unity or unreal, and a step further directly 3d information, though rendering it wouldn't be the easiest thing.
On the flip side when a game engine is already established doing it even remotely like this stops making sense since it's a lot easier to make things work with the implemented systems than to do it yourself.
Making what is my draw maze function a pure collision handler would make a lot of sense along with a component that could handle pickups if that isn't done by who's picking it up.

What should have been done differently

Structure has become horrible with time. I excel in making universal classes that can be used by everything instead of moving things to where they really should be. I also shouldn't be afraid to rewrite a class when realizing this (which I obviously was). A good example is the character class. The only difference between an enemy and the player is the initialization and update script. Everything is stored and handled in the character class when it really shouldn't. The enemy don't need a score when the idea of having it drop collectibles on death was discarded since they aren't actively killed, and the movement although not a bad idea originally ballooned out of control when it had to handle collision detection due to my generation system. The UI should most definitely never have been on the character but in a game handling class or ui class (preferably the latter).

The menus are not too bad, they have a fair bit of repeating code that in retrospect been condensed a fair bit and optimized but at the very least it is quite clean since it's mostly just printing to screen.

The map printing is in my opinion good, the only major flaw besides the enemy handling being tacked on is that the printing should have grabbed the icons from a single list as I have a habit of making multiple identical list which is bad for changes or scaling.

Something that really should have been done is removing old unused code as new was introduced. Instead things ended up left in comment bubbles and weaved into things it has nothing to do with making it just that much harder to remove.

What have I learned from this?

-No matter how much I hate it making diagrams in advance is helpful in the long run.
-The moment you see a repeat try to condense it instead of just thinking that it will only be these two lines. It quickly becomes these three, four, fifteen lines and suddenly it’s a monster.
-Don’t include libraries. It’s counter intuitive to making it mesh but really try making it work another way before doing so. Dependencies quickly spiral out of control.
-Don’t be afraid to start over. Spaghetti code is fun and easy to mesh but impossible to scale. Things take longer and it’s ultimately more satisfying with clean code.
-Don’t scrap an idea just because you can’t figure it out, come back later and work on something different until you either figure out a better way or have to give up to progress.
-You find easier and better ways from optimizing so design optimize and start from scratch with a much better way when you see it instead of trying to fix a mistake.
-Finish what you are working on before adding to it, adding without completing makes a mess.

-Summarized focus on structure and simplicity over just working and writing nonstop.